

Blockchain Implementation for Web-Based Personal Digital Document Verification

1st Wisnu Uriawan
Informatics Department
UIN Sunan Gunung Djati Bandung
West Java, Indonesia
wisnu_u@uinsgd.ac.id

2nd Syahir Mohamad Ramdhan
Informatics Department
UIN Sunan Gunung Djati Bandung
West Java, Indonesia
syahirmr125@gmail.com

3rd Salma Nur Oktavia
Informatics Department
UIN Sunan Gunung Djati Bandung
West Java, Indonesia
salmanuroktavia2004@gmail.com

4th Rizkia Nuari Fujiana
Informatics Department
UIN Sunan Gunung Djati Bandung
West Java, Indonesia
rizkianuari83@gmail.com

5th Yan Syafiq Albari
Informatics Department
UIN Sunan Gunung Djati Bandung
West Java, Indonesia
albarisyafiq04@gmail.com

Abstract—

Index Terms—Blockchain, SHA-256, document integrity, digital document verification, web application, smart contract

I. INTRODUCTION

The use of personal documents in digital form has become a fundamental requirement in various administrative services, including government, education, and business sectors. This transformation is closely related to the urgency of privacy protection in the digital era, where the distribution of critical identity documents such as the Indonesian Identity Card (*Kartu Tanda Penduduk* or *KTP*) is increasingly restricted and is no longer recommended to be physically photocopied merely for administrative purposes in order to prevent identity theft and data leakage [1]. On the other hand, conventional document authenticity verification processes, such as physical legalization mechanisms, are becoming increasingly inefficient. Manual legalization often requires a long processing time, remains vulnerable to forged official stamps, and creates practical difficulties for the public, thereby encouraging the integration of cryptographic technology as an alternative to physical validation mechanisms [2].

However, this digitalization also introduces its own challenges. Incidents of data manipulation and digital document forgery still frequently occur. Personal documents can be copied, modified, or falsified relatively easily without leaving directly visible traces of alteration. Cases involving the falsification of critical documents often occur due to weak security mechanisms in conventional database systems. Such manipulation not only has the potential to cause financial losses, but also reduces public trust in institutional services. This condition makes the authenticity and integrity of personal documents difficult to ensure without an adequate verification mechanism [3], [4]. In other words, the wider the use of personal digital documents becomes, the greater the urgent need to maintain both security and trust in those documents.

The issue of digital document integrity cannot be addressed merely by providing easily accessible storage media. A system must also be able to ensure that stored or verified documents have not been altered. In practice, many document storage systems still rely on centralized databases (*centralized database*). This approach is indeed simple and easy to implement, but it carries significant risks. If the server experiences failure, administrator accounts are misused, or internal manipulation occurs, data integrity may be compromised. Such weaknesses are often associated with the problem of a *single point of failure*; therefore, a more transparent and distributed approach is required, one that does not fully depend on a single managing authority [5].

One technology that can be used to address this problem is *blockchain*. Conceptually, the fundamental idea of *blockchain* is related to the *time-stamping* mechanism, namely the recording of the time of a data item so that its existence and authenticity can be proven [6]. In its development, *blockchain* has become known as a distributed ledger that supports transparent, tamper-resistant, and traceable data recording. These characteristics make *blockchain* relevant for digital document verification processes. Nevertheless, storing complete personal documents directly on a *blockchain* network is neither a wise nor an efficient option. In addition to increasing storage requirements and computational costs, recording personal data transparently on a public ledger (*public ledger*) would result in serious privacy violations.

To overcome this issue, documents can be represented in the form of *hash* values. Cryptographic functions such as SHA-256 can generate a unique digital fingerprint from the content of a document. This *hash* value is then recorded in a *smart contract* on the *blockchain*, rather than storing the document file directly [7], [8]. With this approach, the privacy of data owners remains fully protected because the system can verify document authenticity without exposing the contents of personal documents on the *blockchain* network [9].

In addition, even a minor change to a document will produce a different *hash* value, allowing indications of manipulation to be precisely identified through the *hash* comparison process.

Several previous studies have discussed the use of *blockchain* to maintain digital document integrity in various contexts. In office documents, *blockchain* has been used to help detect unauthorized document modifications [10]. In electronic archive management, this technology has been utilized to preserve archival data integrity [11]. In addition, file integrity monitoring in *cloud* environments has also been developed with the support of *smart contracts* [12]. Other studies show that similar approaches can be applied to digital diploma verification, web-based document management systems, and digital content authenticity verification [13]–[15]. From these studies, it can be seen that *blockchain* has strong potential to support data integrity.

Nevertheless, there remains room for developing a simpler and easier-to-use system. Several studies still focus on specific domains, such as academic documents, electronic archives, or systems involving additional components such as IPFS and OCR. On the other hand, studies that directly present a simple web prototype with a workflow consisting of document upload, *hash* generation, *hash* recording on the *blockchain*, and document verification remain relatively limited. Some studies also remain at the conceptual level [16] or focus on the use of *blockchain* for *timestamping* [17]. This indicates that the development of a lightweight, practical, and easily adaptable prototype is still necessary.

Based on this background, this study aims to design and implement a web-based prototype of a personal digital document integrity verification system that utilizes SHA-256 and *blockchain* to ensure document security and integrity. The developed system has a main workflow consisting of document upload, *hash* value generation, *hash* recording on the *blockchain*, and document integrity verification. Through this approach, this study is expected to contribute a personal digital document verification system that is simple, measurable, and capable of maintaining data confidentiality for use in various administrative service needs [18].

The remainder of this paper is structured as follows. Section II discusses relevant previous studies. Section III explains the research methodology and system design used in this study. Section IV presents the implementation results and discussion. Finally, Section V contains the conclusion and directions for future research.

II. RELATED WORK

Studies on the use of *blockchain* for digital document verification have been widely conducted, although their research directions are not always the same. Some studies mainly focus on academic documents, such as certificates, diplomas, and transcripts. Abdelmagid *et al.* proposed a *blockchain*-based academic certificate authentication framework to improve document security and validity [19]. Ayub Khan *et al.* also developed an *Educational Blockchain* architecture to

support diploma tracking and attestation processes in higher education environments [20].

Other studies in the same domain have attempted to strengthen security and privacy aspects in the verification process. Saleh *et al.* introduced a Hyperledger Fabric-based protocol to preserve privacy in academic certificate verification [21]. Meanwhile, Berrios Moya *et al.* applied a *zero-knowledge proof* approach to improve data protection in academic record verification [22]. Rustemi *et al.*, along with another study on a *blockchain*-based digital diploma verification model, also demonstrated that this technology can be used to preserve the authenticity of educational documents [13], [23]. Nevertheless, these studies are mostly directed toward specific academic document needs, so their implementation may not be directly suitable for broader digital document verification requirements.

Beyond the academic context, *blockchain* has also begun to be used in digital document and archive management. Azzam *et al.* utilized a combination of *blockchain* and OCR in an *e-government* environment to maintain the integrity of incoming documents [24]. Sarang *et al.* developed a document management system that combines *blockchain* and IPFS to support digital document management and verification [14]. In addition, Nizamuddin *et al.* used Ethereum and IPFS to build a decentralized document *version control* mechanism [25].

Several other studies discuss more specific contexts. Zhai *et al.* proposed a trusted verification scheme for office documents [10]. Permatasari *et al.* applied *blockchain* to maintain the integrity of electronic archives [11]. Pinheiro *et al.* showed that file integrity in a *cloud* environment can be continuously monitored with the support of *smart contracts* [12]. Avelino and Rocha also demonstrated that a *blockchain*-based approach can be used to verify the authenticity and integrity of web-based content [15]. From these studies, it can be seen that *blockchain* is not only relevant to one type of document, but can also be applied to various digital data verification needs.

To clarify the position of this study in relation to previous works, several of the most relevant studies are summarized in Table I.

Based on Table I, previous studies have provided a strong technological foundation. However, their implementation directions are still largely dominated by institutional domains, such as academic documents including diplomas and certificates, large-scale electronic archives, and *e-government* governance. In addition, several approaches require complex architectures that integrate external components such as IPFS or OCR. From this mapping, a significant *research gap* can be observed in the domain of individual-level civil society. Studies that specifically target the protection and integrity verification of **personal documents**, such as the Indonesian Identity Card (*Kartu Tanda Penduduk* or KTP), Family Card (*Kartu Keluarga* or KK), or Birth Certificate, using a lightweight web architecture that is easily accessible to the public remain very limited.

From a technical perspective, several studies show that the core of *blockchain*-based document verification does not

TABLE I
SUMMARY OF RELATED STUDIES

Author	Focus	Technology	Finding	Limitation
Abdelmagid <i>et al.</i> [19]	Academic certificates	<i>Blockchain</i>	Improves the security, authenticity, and validity of academic certificates through data recording on a distributed network.	Limited to the context of academic documents and does not discuss implementation for general documents.
Ayub Khan <i>et al.</i> [20]	Diploma verification	<i>Blockchain</i>	Supports historical tracking and digital diploma attestation processes with better transparency.	Focused on higher education environments, so the scope of use remains limited.
Azzam <i>et al.</i> [24]	<i>E-government</i> documents	<i>Blockchain</i> , OCR	Maintains the integrity of incoming documents by combining document identification and <i>blockchain</i> -based recording.	Requires OCR integration and adaptation to document characteristics in a specific domain.
Nizamuddin <i>et al.</i> [25]	Document <i>version control</i>	Ethereum, IPFS	Provides a decentralized and auditable mechanism for tracking document changes.	The system architecture is relatively complex because it involves several distributed components.
Abidin and Munggaran [5]	Digital document verification	SHA-256, <i>Blockchain</i>	Records SHA-256-based document <i>hash</i> values on a <i>blockchain</i> network to support document integrity verification.	Does not present a comprehensive and user-oriented web application implementation.
Chaniago <i>et al.</i> [7]	Diploma and transcript verification	<i>Smart Contract</i> , Ethereum	Utilizes <i>hash</i> values and <i>smart contracts</i> to validate and authenticate academic documents.	Focused on academic documents and has not been evaluated on other document types.
Zhai <i>et al.</i> [10]	Office documents	<i>Blockchain</i>	Detects changes in office documents by utilizing the immutability characteristic of <i>blockchain</i> .	The usage context is still limited to certain office environments.
Pinheiro <i>et al.</i> [12]	File integrity in the <i>cloud</i>	<i>Blockchain</i> , <i>Smart Contract</i>	Continuously monitors file integrity and provides a <i>blockchain</i> -based audit mechanism.	More focused on file integrity monitoring than web-based document verification.

require storing the complete document on the network. A more efficient approach is to use a *hash* value as the document representation and then record that value on the *blockchain*. Abidin and Munggaran showed that digital documents can be verified by generating a *hash* value using SHA-256 and then recording it on a *blockchain* network [5]. A similar approach was also used by Chaniago *et al.* in diploma and transcript verification using Ethereum *smart contracts* [7]. This finding is important because it shows that document integrity can be maintained without storing the document content directly on the *blockchain*. If the document content changes, the resulting *hash* value will also change, allowing the modification to be detected.

At a broader level, Zheng *et al.* explained that *blockchain* has characteristics that support data security, such as decentralization, *immutability*, transparency, and auditability [16]. However, the implementation of *blockchain* still faces several challenges. Zheng *et al.*, as well as Morar and Popescu, highlighted issues such as scalability, computational cost, implementation complexity, and integration with existing systems [26], [27]. Therefore, the design of a digital document verification system should not only pursue security aspects but also consider architectural simplicity and ease of implementation.

Based on this discussion, this study takes a more specific and practical position to fill the gap in individual data protection. Unlike previous systems that are complex and focused on macro-level institutions, this study is directed toward developing a simple *web*-based prototype specifically applied to the verification of **personal digital documents**. Since personal documents contain highly sensitive demographic data, this sys-

tem is built purely using a combination of SHA-256 and *smart contracts* with two main objectives: ensuring that documents are not damaged or manipulated, while also guaranteeing that none of the original document content is exposed on a public network. Through this approach, this study is expected to provide measurable security and verification convenience for broader public administrative needs.

III. METHODOLOGY

This study adopts an experimental-implementative approach because the research does not stop at conceptual design, but continues with the development of a testable prototype. The prototype developed in this study is a *web*-based integrity verification system for personal digital documents. This system utilizes SHA-256 to generate a *hash* value from a personal document, and then records the *hash* value on the *blockchain* as reference data for the verification process without exposing the sensitive data contained in the document.

In its implementation, the prototype is developed using SvelteKit and TypeScript at the *web* application layer. The backend is built using SvelteKit API Routes, while the *hash* generation process is performed using the `crypto` module from Node.js. For the *blockchain* component, this study uses a Solidity-based *smart contract* deployed on the Hardhat Local Network. Communication between the backend and the *smart contract* is handled using the `ethers.js` library. With this structure, the system can be tested directly through document registration and document verification workflows [5], [7], [28], [29].

A. Research Design

The initial stage of this study was carried out by identifying the main problems in the digital management of personal documents, such as KTP, KK, or diplomas, particularly those related to data integrity and privacy. Digital documents are easy to store and distribute, but illegal changes to their contents are often not directly visible. This problem becomes highly risky when the system still relies on a centralized mechanism; if a *database* breach or misuse of access rights by internal parties occurs, the risk of *identity theft* becomes very high. Therefore, this study identifies an urgent need for a verification mechanism that is transparent, difficult to manipulate, and still able to maintain the confidentiality of the owner's demographic data.

After the problem was formulated, a literature study was conducted on previous research discussing digital document verification, *hash* functions, *smart contracts*, and the application of *blockchain* in document systems [5], [7], [13], [24]. The results of this study were used as the basis for determining the workflow of the developed prototype. The main workflow of the system consists of document registration, SHA-256 *hash* value generation, *hash* recording on the *blockchain*, and document integrity verification.

This design was chosen because it is simpler and aligned with the objective of the study. The system does not store the original document file on the *blockchain*. The document is only used as input to generate a *hash* value. The data recorded on the *blockchain* consists of the *document ID*, file name, *hash* value, *timestamp*, and uploader address. In this way, the system can still check document changes without storing the document content directly on the *blockchain* network.

B. Agile Development Method

The prototype development was carried out using the Agile approach. This approach was chosen because the Agile method has been proven to be flexible, fast, and efficient in accommodating changes in system requirements during the development process, especially in the design of *web*-based applications and the integration of an *Application Programming Interface* (API) [30], [31]. The system development process was conducted gradually and required several adjustments, especially in the integration between the SvelteKit interface, the *backend*, and the *smart contract*. In this study, Agile is used as a framework to organize the development process, starting from requirement analysis, system design, prototype implementation, *blockchain* integration, system testing, to evaluation and improvement.

The first stage is *requirement analysis*. At this stage, the main functions that must be available in the system are determined, namely receiving document uploads, generating SHA-256 *hash* values, recording *hash* values on the *blockchain*, and verifying document integrity. These requirements are prepared based on the research problem and the results of the literature study.

The second stage is *system design*. At this stage, the system architecture, process flow, and main interface design are prepared. The defined components include the *web* application,

backend API, *hash* generation module, *smart contract*, and local *blockchain* network. This design is needed so that each part of the system has a clear function and can be connected to one another.

The third stage is *prototype implementation*. At this stage, the core modules of the system are gradually developed. The implementation includes the document registration page, document verification page, *hash* generation module, registration API, verification API, and validation result display. The development is carried out by ensuring that the features remain aligned with the main workflow of the study.

The fourth stage is *blockchain integration*. At this stage, the application is connected to the *smart contract* using ethers.js. The *smart contract* is developed using Solidity and deployed on the Hardhat Local Network. This integration enables the system to record document *hashes*, retrieve reference *hashes*, and check the existence of a *document ID* on the local *blockchain* network.

The fifth stage is *system testing*. Testing is conducted to ensure that the registration and verification workflows run according to the design. The tested scenarios include registering an original document, verifying an original document without modification, verifying a modified document, and verifying an unregistered *document ID*.

The final stage is *evaluation and improvement*. At this stage, the development and testing results are reviewed to identify which parts still need improvement. This evaluation helps refine the prototype so that the system becomes more stable and remains aligned with the objectives of the study.



Fig. 1. Agile development stages in the digital document verification system prototype

C. System Architecture

The system architecture is designed in a modular manner so that each component has a clear role. In general, the system consists of users, a *web* application, *backend API*, *hash* generation module, *smart contract*, local *blockchain* network, and

integrity verification output. This separation of components is made so that the document upload, *hash* generation, data recording, and verification processes can run in a structured manner.

Users interact with the system through a *browser*. On the interface side, the system is built using SvelteKit and provides several main pages, namely the home page, document registration page, document verification page, testing history page, and system information page. This interface is used to receive input from users and display the results of the system process.

The backend layer is built using SvelteKit API Routes. The two main endpoints used are `/api/register` and `/api/verify`. The `/api/register` endpoint handles the document registration process, starting from receiving the file, generating the *hash* value, creating the *document ID*, sending data to the *smart contract*, and returning the recording information. Meanwhile, the `/api/verify` endpoint is used to receive the *document ID* and test document, generate a new *hash*, retrieve the reference *hash* from the *blockchain*, and determine the document validity status.

The *hash generation* component is responsible for generating the SHA-256 *hash* value from the uploaded document. This process is performed on the server side using the `crypto` module from Node.js. The *hash* value is used as a unique representation of the document content. Since SHA-256 is deterministic, the same document will produce the same *hash* value, while even a small change in the document will produce a different *hash* value [28].

The *smart contract* component is developed using Solidity. The *smart contract* stores document data in a structure containing the file name, document *hash* value, *timestamp*, uploader address, and data existence status. The main functions used include `registerDocument`, `getDocumentHash`, `verifyDocument`, `getDocument`, and `documentExists`. These functions are used to support the data recording process and the retrieval of reference *hashes* during verification [7], [12].

The *blockchain* network used in this prototype is the Hardhat Local Network [32]. A local network was chosen because it is easier to control during the development and testing process. In addition, the use of a local network allows transactions to be tested without relying on a public network. In this prototype, the original document is not recorded or stored on the *blockchain*. The data recorded *on-chain* only consists of the *document ID*, dummy file name, *hash* value, *timestamp*, and uploader address. This approach not only makes the system more computationally efficient, but also serves as an essential privacy protection layer. Considering the transparent and publicly accessible nature of *blockchain*, storing complete personal documents on the network would be equivalent to exposing sensitive data. By storing only the *hash*, the system can still accurately detect document changes without violating user privacy rights [5], [14].

The integrity verification output is displayed to the user after the *hash* comparison process is completed. If the *hash* value

of the test document is the same as the reference *hash* on the *blockchain*, the system displays the status *VALID*. Conversely, if the *hash* value is different, the system displays the status *INVALID*. This status becomes the basis for determining whether the document is still the same as the previously registered document or has been modified.

D. Implementation Flow

The system implementation flow is divided into two main processes, namely document registration and document verification. These two processes are interrelated. The registration process is used to generate reference data in the form of a document *hash* value, while the verification process uses that reference data to check whether the tested document is still the same as the registered document.

In the registration process, users upload a document through the registration page. The uploaded file is received by the `/api/register` endpoint in the form of *FormData*. After that, the system reads the file as a *buffer* and generates a SHA-256 *hash* value using the `crypto` module. The *hash* value is then used as a unique representation of the document without exposing its original content [33].

After the *hash* value is obtained, the system creates a *document ID* using the *timestamp* and part of the *hash* value. Next, the *document ID*, file name, and *hash* value are sent to the `registerDocument` function in the *smart contract*. When the transaction has been processed on the Hardhat Local Network, the system returns information in the form of the *document ID*, file name, *hash* value, *transaction hash*, and block number.

In the verification process, users enter the *document ID* and upload the document to be tested. The data is received by the `/api/verify` endpoint. The system then generates a new *hash* value from the test document. Before the comparison is performed, the system first checks whether the *document ID* is registered in the *smart contract* through the `documentExists` function.

If the *document ID* is found, the system retrieves the reference *hash* using the `getDocumentHash` function. The new *hash* value is then compared with the reference *hash*. If both values are the same, the document is declared *VALID*. If they are different, the document is declared *INVALID* because it is indicated to have been modified [5], [7], [10].

ARSITEKTUR SISTEM VERIFIKASI DOKUMEN DIGITAL

Gambaran Umum Arsitektur Sistem



Setiap perubahan pada dokumen akan menghasilkan nilai hash yang berbeda, sehingga integritas dokumen dapat diverifikasi.

Fig. 2. Architecture of the digital document verification system

RANCANGAN ALUR IMPLEMENTASI SISTEM INTEGRITAS DOKUMEN BERBASIS BLOCKCHAIN

Registrasi dokumen dan verifikasi integritas berbasis hash dan blockchain

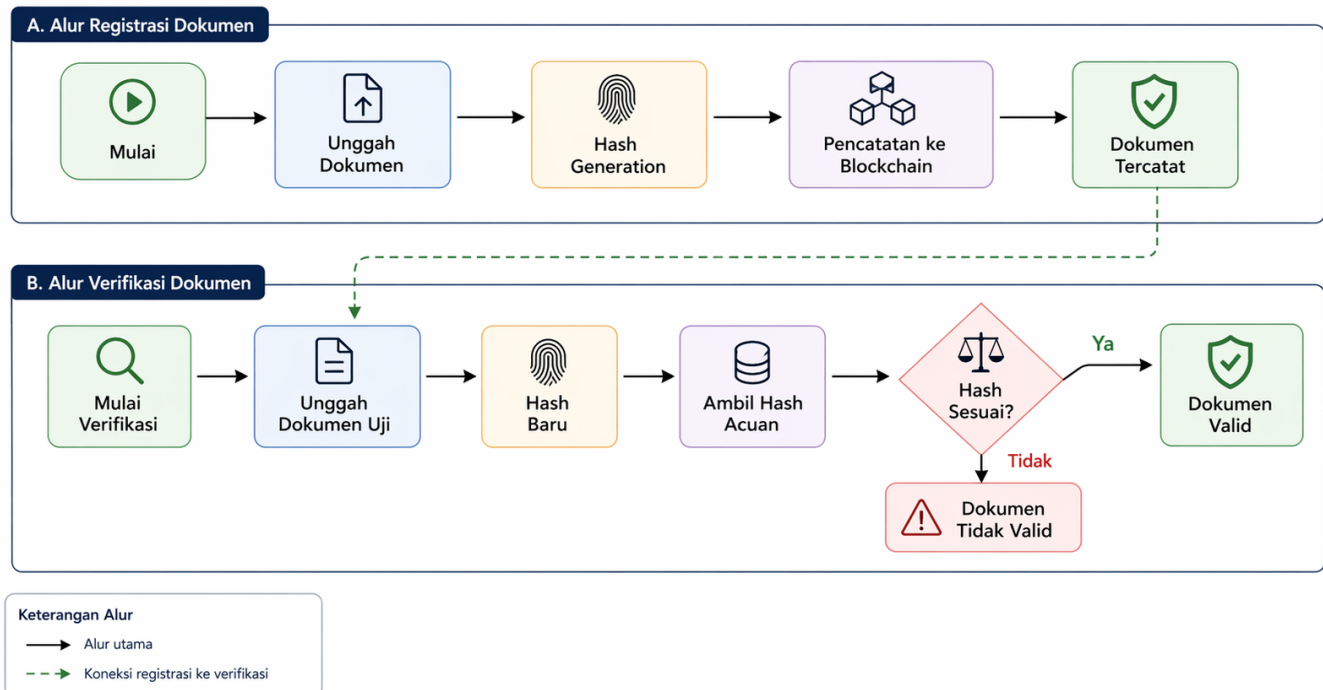


Fig. 3. Implementation flow of document registration and verification

E. Algorithm Design

The system algorithm is designed to support two main processes, namely document registration and document verification. In the registration process, the system receives a document file, generates a SHA-256 *hash* value, creates a *document ID*, and then records the document data to the *smart contract*. In the verification process, the system receives a *document ID* and a test document, generates an *input hash*, retrieves the *stored hash* from the *blockchain*, and then compares the two values to determine the document status.

Algorithm 1 Document Registration and Verification

```
Registration input: document
Registration output: document ID, document hash, transaction hash, block number
Registration process:
buffer ← read_file(document)
document_hash ← SHA-256(buffer)
document_id ← create_document_id(document_hash, timestamp)
tx ← registerDocument(document_id, file_name, document_hash)
receipt ← wait_transaction(tx)
return document_id, document_hash, tx.hash, receipt.blockNumber
Verification input: document_id, test_document
Verification output: input_hash, stored_hash, status
Verification process:
test_buffer ← read_file(test_document)
input_hash ← SHA-256(test_buffer)
exists ← documentExists(document_id)
if exists == false then
    status ← document_id_not_found
else
    stored_hash ← getDocumentHash(document_id)
    if input_hash == stored_hash then
        status ← VALID
    else
        status ← INVALID
    end if
end if
return input_hash, stored_hash, status
```

The algorithm shows that the verification process is not performed by reading or directly comparing the content of the document. The system only compares the document representation in the form of a *hash* value. This approach makes the verification process lighter, but still able to detect changes in the document because any change in the content will produce a different *hash* value [15], [28].

F. Testing Scenario

Functional testing is conducted to ensure that the registration and verification workflows run according to the design. The focus of testing is on the system's ability to generate *hash* values, record them on the *blockchain*, retrieve the reference data, and distinguish between original and modified documents. All testing is conducted in a local environment using the Hardhat Local Network.

The first scenario is the registration of an original personal document. In this scenario, the test does not use real demographic data, but is represented using a dummy personal document file, for example *KTP_Dummy.pdf*, to comply with privacy testing ethics. The system is tested to ensure that the document can be uploaded locally, the SHA-256 *hash* value can be generated on the client side, and the document metadata can be recorded in the *smart contract*.

The examined outputs include the *document ID*, file name, *hash* value, *transaction hash*, and block number.

The second scenario is the verification of the original document without modification. This test is conducted to ensure that the same document produces a *hash* value identical to the reference *hash* recorded on the *blockchain*. If both values are the same, the system should display the *VALID* status.

The third scenario is the verification of a modified document. At this stage, changes are made to the content or structure of the test document. After that, the system generates a new *hash* value and compares it with the reference *hash*. If the *hash* value is different, the system should display the *INVALID* status.

The fourth scenario is verification using an unregistered *document ID*. This scenario is required to ensure that the system can handle invalid input. If the *document ID* is not found on the *blockchain*, the system displays an error message stating that the data was not found.

In addition to these four scenarios, the system also provides a testing history feature on the user interface side. This history is stored locally using *localStorage* in the *browser*. This feature is only used for prototype demonstration purposes, not as a permanent database [34]. Through the *smart contract* implementation validated in this testing, transparency and data resistance against interference can be ensured [35]. With this set of tests, the prototype can demonstrate that document integrity can be verified through the comparison of *hash* values recorded on the *blockchain*.

IV. RESULT AND DISCUSSION

A. Result

This section explains the implementation and testing results of the *blockchain*-based digital document verification system prototype. Testing was conducted to determine whether the main workflow of the system could run according to the design, starting from document registration, *hash* value generation, *hash* recording on the *blockchain*, and document verification. In general, the prototype was able to perform two main functions, namely document registration and document verification.

In the registration process, the system generates a SHA-256 *hash* value from the uploaded document and then records the value into the *smart contract*. In the verification process, the system regenerates the *hash* value from the test document and compares it with the reference *hash* previously recorded on the *blockchain*. If both *hash* values are identical, the document is declared valid. If they are different, the document is considered to have been modified.

This prototype was built using SvelteKit as the *web* application, SvelteKit API Routes as the backend layer, and the Node.js *crypto* module to generate SHA-256 *hash* values. In the *blockchain* component, the system uses a Solidity *smart contract* deployed on the Hardhat Local Network. Communication between the backend and the *smart contract* is performed using ethers.js. In this implementation, the original document is not stored on the *blockchain*. The recorded data

only consists of the *document ID*, file name, *hash* value, *timestamp*, and uploader address.

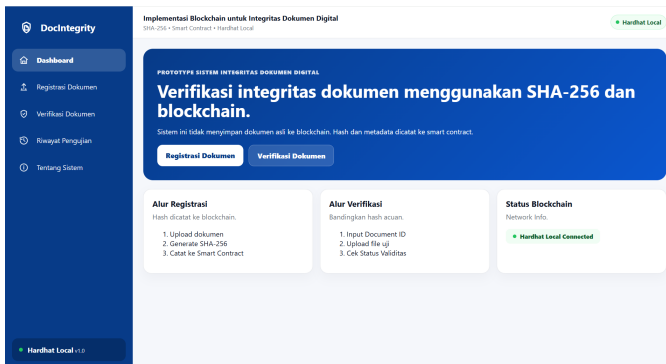


Fig. 4. Prototype system dashboard

1) *Document Registration Results*: At the registration stage, the user uploads a document through the document registration page. After the document is received by the system, the backend reads the file and generates a SHA-256 *hash* value. The system then creates a *document ID* as the document identifier, and records the *document ID*, file name, and *hash* value into the *smart contract*. After the recording process is completed, the system displays the registration result, including the *document ID*, file name, *hash* value, *transaction hash*, and block number.

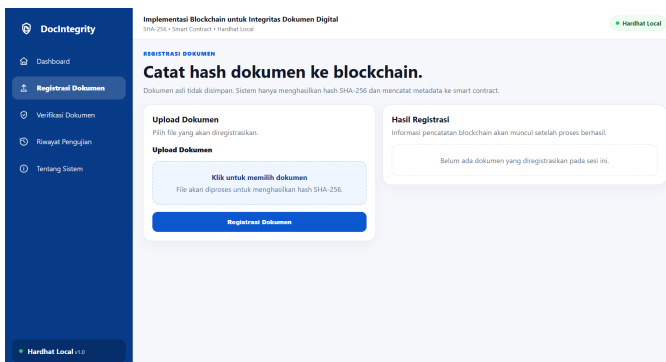


Fig. 5. Document registration page

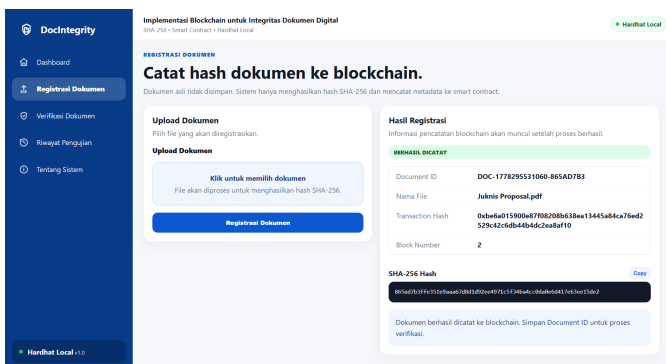


Fig. 6. Document registration result and hash recording on the blockchain

The result at this stage shows that the system is able to generate a *hash* value from a digital document and record the data on the local *blockchain*. The *transaction hash* and block number displayed by the system serve as evidence that the recording transaction has been processed on the Hardhat Local Network.

2) *Document Verification Results*: At the verification stage, the user enters the *document ID* and uploads the document to be tested. The system then generates a new *hash* value from the document. After that, the system retrieves the reference *hash* from the *blockchain* based on the entered *document ID*. Both *hash* values are then compared to determine the document status.

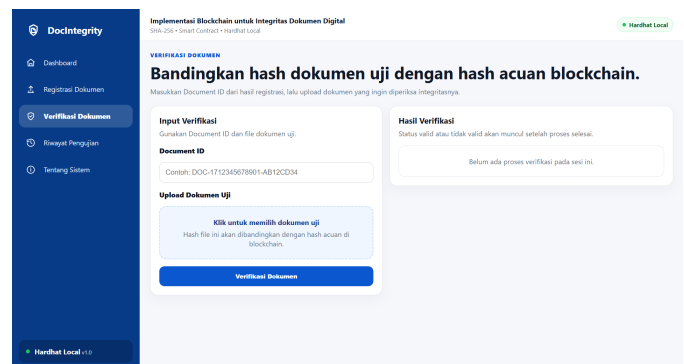


Fig. 7. Document verification page

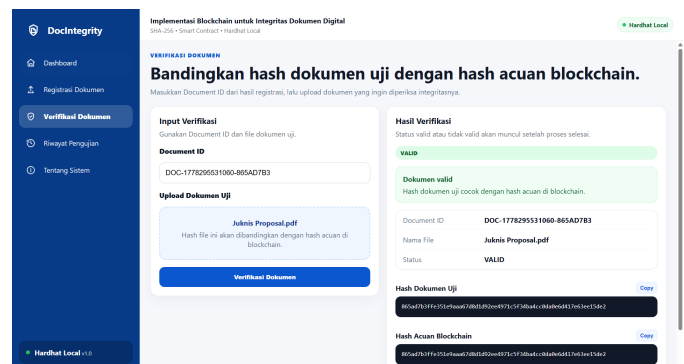


Fig. 8. Verification result of the original document with valid status

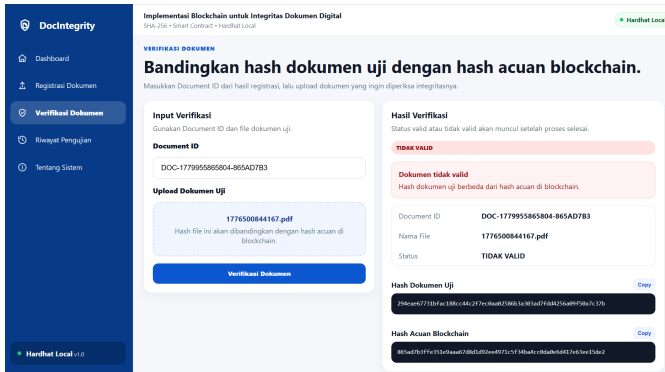


Fig. 9. Verification result of the modified document with invalid status

The verification result shows that the original document produces the same *hash* value as the reference *hash*, so the system displays the *VALID* status. Conversely, when a modified document is used as input, the generated *hash* value differs from the reference *hash*. In this condition, the system displays the *INVALID* status. This result shows that changes in a document can be detected through differences in *hash* values.

3) *System Testing Results:* Testing was conducted based on several scenarios described in the methodology section. These scenarios include original document registration, original document verification, modified document verification, and verification using an unregistered *document ID*. The summary of the testing results is shown in Table II.

Based on Table II, all testing scenarios ran according to the expected outputs. The system was able to record document *hashes* on the *blockchain*, retrieve the reference *hash*, and distinguish between original and modified documents. In addition, the system was also able to handle the condition when a user entered an unregistered *document ID*.

To strengthen the testing results, Fig. 10 shows the transaction log on the Hardhat Local Network when the system executed the *registerDocument* function in the *smart contract*. The log displays information such as the *transaction hash*, sender address, *smart contract* address, gas usage, and block number. This information indicates that the document data recording process has been processed on the local *blockchain* network.

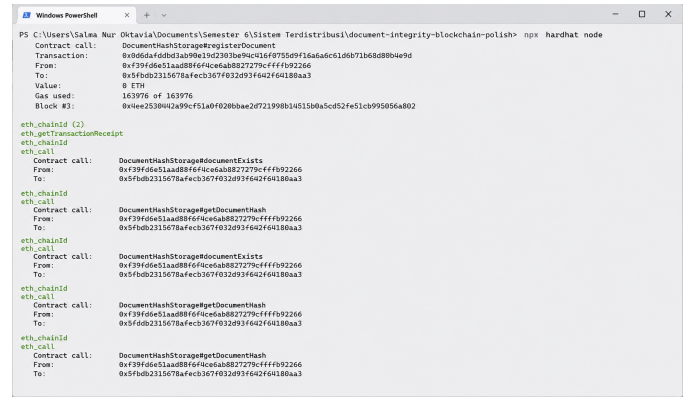


Fig. 10. Transaction log of the *registerDocument* function on the Hardhat Local Network

B. Discussion

The implementation results show that the combination of the SHA-256 *hash* algorithm and *blockchain* technology can be used to support the verification process of digital document integrity. In the developed system, the original document is not stored directly on the *blockchain*. The system only records the *hash* value and supporting metadata, such as the *document ID*, file name, *timestamp*, and uploader address. This approach is consistent with the principle of *hash*-based document verification, in which a document is represented as a digital fingerprint so that the verification process does not need to store the full document content [5]. The SHA-256 standard itself is used as a cryptographic *hash* function that can generate a fixed representation of digital data [28].

The use of SHA-256 in this system plays an important role because the algorithm is deterministic. This means that the same document will produce the same *hash* value, while even a small change in the document content will produce a different *hash* value [28]. The testing results show that the original document produces the same *hash* value as the reference *hash* on the *blockchain*, so the system displays the *VALID* status. Conversely, the modified document produces a different *hash* value and the system displays the *INVALID* status. This behavior confirms that the proposed mechanism is able to detect document alteration without requiring direct inspection of the document contents.

It is crucial to note that in the verification of **personal documents**, this approach represents an essential security standard. Considering the transparent nature of *blockchain* as a *public ledger*, storing original document content, such as KTP or KK, directly *on-chain* would create a high privacy risk, including the potential for *doxxing*. By recording only the *hash* representation in the *smart contract*, the system is able to validate data integrity without exposing any sensitive content to the public network. This approach is aligned with the principle of personal data protection, which requires the confidentiality of sensitive information in digital verification systems [1], [2]. This result is also in line with the study by Abidin and Munggaran, which utilized SHA-256 to support *blockchain*-based digital document verification [5].

TABLE II
SYSTEM TESTING RESULTS

No	Scenario	Input	Testing Result	Status
1	Original document registration	Original file	The system successfully generated the <i>document ID</i> , <i>hash</i> value, <i>transaction hash</i> , and block number after the document was registered on the <i>blockchain</i> network.	Success
2	Original document verification	<i>Document ID</i> and original file	The generated <i>hash</i> matched the value stored on the <i>blockchain</i> ; therefore, the system displayed the <i>VALID</i> status.	Success
3	Modified document verification	<i>Document ID</i> and modified file	The generated <i>hash</i> differed from the value stored on the <i>blockchain</i> ; therefore, the system displayed the <i>INVALID</i> status.	Success
4	Unregistered <i>Document ID</i> verification	Unavailable <i>Document ID</i>	The system displayed a message stating that the <i>Document ID</i> was not found on the <i>blockchain</i> network.	Success

The role of *blockchain* in this system is to serve as a medium for recording reference *hashes* that are more difficult to manipulate compared to conventional centralized storage. The concept of recording data to prove the existence and integrity of a document is related to the digital *timestamping* mechanism introduced by Haber and Stornetta [6]. In this prototype implementation, the *smart contract* is used to record and retrieve the reference *hash* based on the *document ID*. The use of *smart contracts* as a document verification medium has also been applied in the study by Chaniago et al. in the context of Ethereum-based academic document verification [7].

The testing results show that the *smart contract* can support document registration and verification processes in a structured manner. At the registration stage, the *registerDocument* function is used to record the *document ID*, file name, and *hash* value on the local *blockchain* network. Information such as the *transaction hash*, gas usage, and block number on the Hardhat Local Network indicates that the recording process has been processed as a *blockchain* transaction. At the verification stage, the system retrieves the reference *hash* from the *smart contract* and compares it with the *hash* of the test document. A similar mechanism was also found in the study by Pinheiro et al., which utilized *blockchain* and *smart contracts* to monitor file integrity [12].

From the efficiency perspective, storing only the *hash* value is more lightweight than storing the complete document on the *blockchain*. Storing full documents may increase storage requirements, transaction costs, and processing overhead. Therefore, this system only uses the *blockchain* as a medium for recording *hash* values and supporting metadata. A similar approach was also used by Permatasari et al. to maintain the integrity of electronic archives through *blockchain* [11]. In addition, Zhai et al. also showed that a *blockchain*-based document verification scheme can be used to detect changes in office documents [10].

Compared to approaches that use additional components such as IPFS or OCR, this prototype adopts a simpler design. Several previous studies combined *blockchain* with IPFS for document management and version control, as shown in the study by Nizamuddin et al. [25]. Sarang et al. also developed a document management system by combining *blockchain* and IPFS [14]. Meanwhile, Azzam et al. utilized a combination of

blockchain and OCR in the context of *e-government* document management [24]. Compared to those approaches, this study focuses on lighter core functions, namely document registration, *hash* generation, *hash* recording on the *blockchain*, and document integrity verification.

Although the testing results show that the prototype can run according to the research objectives, this system still has several limitations. First, *blockchain* testing is still conducted on the Hardhat Local Network, so it does not yet represent public network conditions such as a *testnet* or *mainnet*. On a public network, factors such as gas fees, transaction confirmation time, network congestion, and connection stability need to be considered. These challenges are in line with the discussion by Zheng et al. regarding *blockchain* architecture, consensus, and development trends [16]. In addition, scalability and performance issues are also important challenges in the implementation of *blockchain* technology [26].

Second, the system has not yet been equipped with user authentication and access control management. In this prototype, document recording still focuses on proving integrity through *hash* values, not on complete user identity management. Third, the activity history in the interface is still used for demonstration purposes and has not been stored in a permanent database. Fourth, the system has not yet provided an *off-chain* document storage mechanism. If the system is developed for real-world use, *off-chain* storage such as IPFS may be considered, especially when documents need to be managed together with their metadata and version history [14], [25].

In addition, the prototype emphasizes privacy-preserving verification for personal documents by avoiding direct exposure of the original file. This aspect is important because personal documents may contain sensitive demographic information, such as identity numbers, addresses, family-related data, and other administrative attributes. By using only the *hash* value as the verification reference, the system can maintain document integrity while reducing the possibility of sensitive data disclosure. Therefore, the proposed approach is not only useful for technical integrity checking, but also relevant for administrative services that require secure and privacy-aware document validation.

Overall, the implementation and testing results show that the developed prototype has been able to prove the main

concept of this study. Digital document integrity can be checked through the process of *hash* generation, reference *hash* recording on the *blockchain*, and *hash* comparison at the verification stage. With this approach, the system can detect document changes without storing the original file on the *blockchain*. This prototype can serve as an initial basis for the development of a more complete digital document verification system, for example by integrating it with a *testnet*, adding user authentication, storing history in a permanent database, and supporting *off-chain* storage.

V. CONCLUSION

This study has successfully designed and implemented a *web*-based prototype for verifying the integrity of **personal digital documents** by utilizing the SHA-256 algorithm and *blockchain* technology. The developed system focuses on two main processes, namely document registration and document verification. In the registration process, the uploaded document is processed to generate a unique SHA-256 *hash* value, which is then recorded on the *blockchain* through a *smart contract*. Meanwhile, in the verification process, the system regenerates the *hash* value from the test document and compares it with the previously recorded reference *hash*.

The testing results show that the prototype is capable of distinguishing between original documents and documents that have been modified with a high level of accuracy. By storing only the *hash* value on the *blockchain* and excluding the original document, this system provides an essential layer of privacy protection, thereby reducing the risk of sensitive data leakage or *doxxing*, which commonly occurs in centralized storage systems. Through this approach, the integrity of personal documents can be verified transparently without compromising the confidentiality of users' demographic data.

Nevertheless, the developed prototype still has several limitations. The current *blockchain* testing is still limited to the Hardhat Local Network. In addition, the system has not yet been equipped with advanced user authentication features, access control management, and a permanent database for more comprehensive document history management.

Future development can be directed toward integration with public *blockchain* networks such as the Ethereum *testnet*, as well as the addition of a digital identity management system, such as Decentralized Identity (DID). Furthermore, the system can be strengthened through the integration of *off-chain* storage such as IPFS for managing larger documents while maintaining data security. With these improvements, this system is expected to become an applicable solution for the public in maintaining the security and authenticity of personal documents in the digital era.

ACKNOWLEDGMENT

The authors would like to express their gratitude to the Informatics Department of UIN Sunan Gunung Djati Bandung for supporting this research.

REFERENCES

- [1] A. M. Baqis and M. I. P. Nasution, "Pentingnya perlindungan dan keamanan data privasi di era digital," *Jurnal Manajemen dan Pendidikan Agama Islam*, vol. 3, no. 3, pp. 396–404, 2025.
- [2] M. Hikmatyar, A. Sudiarjo, and H. Lutfiani, "Model bisnis aspek legal pada keamanan dokumen dan data pribadi e-government," *Jurnal Teknik Informatika dan Sistem Informatika*, vol. 9, no. 1, pp. 633–647, 2022.
- [3] H. P. Fitriani, N. Andriyani, C. Anggraeni, M. F. Ashofwani, and A. M. Miftah, "Analisis penggunaan blockchain untuk pengelolaan di bidang pendidikan," *JATI (Jurnal Mahasiswa Teknik Informatika)*, vol. 9, no. 1, pp. 1041–1045, 2025.
- [4] A. Argani and W. Taraka, "Pemanfaatan teknologi blockchain untuk mengoptimalkan keamanan sertifikat pada perguruan tinggi," *ADI Bisnis Digital Interdisiplin Jurnal*, vol. 1, no. 1, pp. 10–21, 2020.
- [5] M. F. Abidin and L. C. Munggaran, "Blockchain-based digital document verification using sha-256 on the internet computer protocol (icp)," *Journal of Electrical, Electronic, Information, and Communication Technology*, vol. 7, no. 2, pp. 64–68, 2024.
- [6] S. Haber and W. S. Stornetta, "How to time-stamp a digital document," *Journal of Cryptology*, vol. 3, no. 2, pp. 99–111, 1991.
- [7] N. Chaniago, P. Sukarno, and A. A. Wardana, "Electronic document authenticity verification of diploma and transcript using smart contract on ethereum blockchain," *Register: Jurnal Ilmiah Teknologi Sistem Informatika*, vol. 7, no. 2, pp. 149–163, 2021.
- [8] N. D. S. Supriyanto, "Blockchain dalam pendidikan menjamin keamanan dan keabsahan data akademik di era digital," *Jurnal Ilmu Manajemen dan Pendidikan*, vol. 2, no. 1, pp. 524–534, 2025.
- [9] J. P. Azzollini, M. E. García, and N. L. Zubeldia, "Blockchain-based data integrity management system for decentralized cloud computing," *International Journal of Information Engineering and Science*, vol. 1, no. 2, pp. 01–07, 2024.
- [10] X. Zhai, S. Pang, M. Wang, S. Qiao, and Z. Lv, "Tvs: a trusted verification scheme for office documents based on blockchain," *Complex and Intelligent Systems*, vol. 9, no. 3, pp. 2865–2877, 2023.
- [11] I. Permatasari, M. Essaid, H. Kim, and H. Ju, "Blockchain implementation to verify archives integrity on cilegon e-archive," *Applied Sciences*, vol. 10, no. 7, p. 2621, 2020.
- [12] A. Pinheiro, E. D. Canedo, R. d. O. Albuquerque, and R. T. de Sousa Júnior, "Validation of architecture effectiveness for the continuous monitoring of file integrity stored in the cloud using blockchain and smart contracts," *Sensors*, vol. 21, no. 13, p. 4440, 2021.
- [13] A. Rustemi, F. Dalipi, V. Atanasovski, and A. Risteski, "Diar: a blockchain-based system for generation and verification of academic diplomas," *Discover Applied Sciences*, vol. 6, no. 6, p. 297, 2024.
- [14] S. Sarang, D. Rana, S. Patel, D. Savaliya, U. P. Rao, and A. Chaurasia, "Document management system empowered by effective amalgam of blockchain and ipfs," *Procedia Computer Science*, vol. 215, pp. 340–349, 2022.
- [15] M. Avelino and A. A. d. A. Rocha, "Blockproof: a framework for verifying authenticity and integrity of web content," *Sensors*, vol. 22, no. 3, p. 1165, 2022.
- [16] Z. Zheng, S. Xie, H. Dai, X. Chen, and H. Wang, "An overview of blockchain technology: Architecture, consensus, and future trends," in *2017 IEEE International Congress on Big Data (BigData Congress)*. IEEE, 2017, pp. 557–564.
- [17] B. Gipp, N. Meuschke, and A. Gernandt, "Decentralized trusted timestamping using the crypto currency bitcoin," *arXiv preprint arXiv:1502.04015*, 2015.
- [18] M. Fachri and H. Soeparno, "Digital document security system using blockchain technology," *Journal of Information Systems Engineering and Management*, vol. 10, no. 10s, 2025.
- [19] R. Abdelmagid, M. Abdelsalam, and F. K. Alsheref, "A blockchain framework for academic certificates authentication," *International Journal of Advanced Computer Science and Applications*, vol. 15, no. 7, 2024.
- [20] A. Ayub Khan, A. A. Laghari, A. A. Shaikh, S. Bourouis, A. M. Mamlouk, and H. Alshazly, "Educational blockchain: A secure degree attestation and verification traceability architecture for higher education commission," *Applied Sciences*, vol. 11, no. 22, p. 10917, 2021.
- [21] O. S. Saleh, O. Ghazali, and N. B. Idris, "A new privacy-preserving protocol for academic certificates on hyperledger fabric," *International Journal of Advanced Computer Science and Applications*, vol. 14, no. 2, pp. 595–609, 2023.

- [22] J. A. Berrios Moya, J. Ayode, and M. A. Uddin, "A zero-knowledge proof-enabled blockchain-based academic record verification system," *Sensors*, vol. 25, no. 11, p. 3450, 2025.
- [23] Alfina and Syafrinal, "Model sistem verifikasi dokumen ijazah digital berbasis teknologi blockchain," *SMARTICS Journal*, vol. 8, no. 2, pp. 59–65, 2022.
- [24] F. Azzam, M. Jaber, A. Saies, T. Kirresh, R. Awadallah, A. Karakra, H. Barghouthi, and S. Amarneh, "The use of blockchain technology and ocr in e-government for document management: Inbound invoice management as an example," *Applied Sciences*, vol. 13, no. 14, p. 8463, 2023.
- [25] N. Nizamuddin, K. Salah, M. A. Azad, J. Arshad, and M. H. Rehman, "Decentralized document version control using ethereum blockchain and ipfs," *Computers and Electrical Engineering*, vol. 76, pp. 183–197, 2019.
- [26] X. Zheng, Y. Zhu, and X. Si, "A survey on challenges and progresses in blockchain technologies: A performance and security perspective," *Applied Sciences*, vol. 9, no. 22, p. 4731, 2019.
- [27] C. D. Morar and D. E. Popescu, "A survey of blockchain applicability, challenges, and key threats," *Computers*, vol. 13, no. 9, p. 223, 2024.
- [28] National Institute of Standards and Technology, "Secure hash standard (shs)," FIPS PUB 180-4, 2015. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf>
- [29] M. A. Cardenas-Quispe and A. Pacheco, "Blockchain ensuring academic integrity with a degree verification prototype," *Scientific Reports*, vol. 15, no. 1, p. 9281, 2025.
- [30] A. Ariesta, Y. N. Dewi, F. A. Sariasih, and F. W. Fibriany, "Penerapan metode agile dalam pengembangan application programming interface system pada pt xyz," *Jurnal Manajemen Informatika (JAMIKA)*, 2021.
- [31] M. F. Iqbal and H. P. Putro, "Penerapan simple agile methodology dalam pengembangan aplikasi web," *Automata*, 2022.
- [32] G. L. Saroinson, A. Sambul, and S. R. U. A. Sompie, "Implementation of blockchain technology in e-voting systems," *Jurnal Teknik Informatika*, vol. 20, no. 1, pp. 11–18, 2025.
- [33] S. P. Anggraeni, "Penerapan algoritma kriptografi sha-256 dan teknologi blockchain untuk keamanan citra digital," *Laporan Tugas Akhir, Universitas Islam Sultan Agung*, 2025.
- [34] S. Abdullah, "Implementasi blockchain untuk keamanan data akademik dalam sistem informasi perguruan tinggi," *GO INFOTECH: JURNAL ILMIAH STMIK AUB*, vol. 31, no. 1, pp. 149–160, 2025.
- [35] A. Fernando, "Implementasi smart contract untuk e-voting pada jaringan blockchain," Master's thesis, Universitas Buddhi Dharma, 2022.